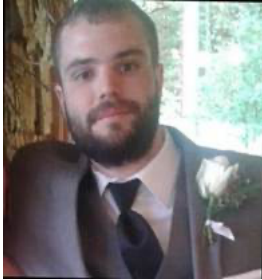
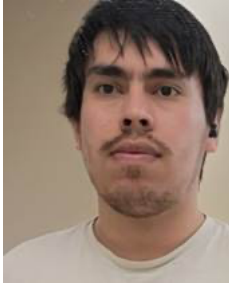


Software Requirements Specification

SP-14 - Green - Chess AI

CS 4850 - Section 03 – Fall 2025

Sep 10, 2025

			
Matt Staats Developer	Jason Nguyen Documentation	Carlos Pena Documentation	Chiagoziem Iteogu Developer

Project Team

Roles	Name	Team Function	Contact (Cell Phone) and Alt Email
Project owner or Advisor	Sharon Perry	Facilitate project progress; advise on project planning and management.	770.329.3895 Sperry46@kennesaw.edu
Team leader	Matt Staats	Team Leader & Developer	404.405.6710 matthewcstaats@gmail.com
Team members	Jason Nguyen	Documentation & Testing	404.747.3352 Neilgroup101@gmail.com
	Carlos Pena	Documentation & Website	404.519.5636 Cromero2511@gmail.com
	Chiagoziem Iteogu	Developer	478.955.1662 Goziem.iteogu@yahoo.com

Table of Contents

1.0 Introduction.....	4
1.1 Overview	4
1.2 Project Scope.....	4
1.3 Definitions and Acronyms	4
1.4 Assumptions.....	4
1.5 References	5
2.0 Constraints.....	5
2.0.1 Guest Mode.....	6
2.0.2 No Mobile.....	6
2.0.3 No Save or Restore Game Points.....	6
2.0.4 Multiplayer Online.....	6
2.0.5 AI Difficulty Scaling.....	6
2.1 Environment.....	6
2.2 System.....	7
3.0 Functional Requirements.....	7
3.1 Game Modes.....	7
3.1.1 User vs User Mode.....	7
3.1.2 User vs AI Mode.....	7
3.2 Game Rules.....	7
3.2.1 Standard Rules.....	7
3.2.2 Special Rules.....	8
3.2.2.1 Castling.....	8
3.2.2.2 Pawn Promotion.....	8
3.2.2.3 En Passant.....	8
3.2.2.2 Double Step Pawn Move.....	8
3.3 Database.....	8
3.3.1 PGN Game State.....	8
3.3.2 Player's Moves.....	8
3.3.3 AI Engine Date.....	8

Table of Contents (Continued)

3.3.4 AI Generated Moves	8
3.4 UI.....	9
3.4.1 Chessboard.....	9
3.4.1.1 Drag and Drop Pieces.....	9
3.4.1.2 Checkmate, Stalemate Detection.....	9
3.4.1.3 Player's Turn Indicator.....	9
3.4.1.4 Win/Loss Score Counter.....	9
3.5 AI Engine.....	9
4.0 Non-Functional Requirements.....	9
4.1 Maintainability.....	9
4.2 Reliability	10
4.3 Usability	10
4.4 Scalability.....	10
5.0 External Interface Requirements.....	10
5.1 User Interface Requirements.....	10
5.2 Hardware Interface Requirements.....	10
5.3 Software Interface Requirements.....	10
5.4 Communication Interface Requirements.....	11
6.0 Analysis.....	11
6.1 Use Cases.....	11
6.1.1 Player vs AI	11
6.1.2 Player vs Player (Local).....	11
6.2 Data Flow	12
6.2.1 Game Flow	12
6.2.2 AI Engine Flow.....	12
6.2.3 Training Flow.....	12

1.0 Introduction

1.1 Overview

The document provides instructions for the requirement of SP-14 AI Chess, enlisting the needed requirements specification to achieve project completion. This document will lay out the following requirements for stakeholders to understand and plan accordingly.

AI Chess project is a full stack development web-based application that utilizes AI to play against user, or player against player. Using open-source framework of React, web API, chessboard.js, Pytorch, and Python-Chess for development.

1.2 Project Scope

Build a functional web application that allows the users to play chess. The application will offer two different game modes: User vs User and User vs AI. The game will follow all the standard rules of chess. While having an interactive UI to move the pieces around the board allowing users to drag and drop or click and place.

1.3 Definitions and Acronyms

- PGN – Portable Game Notation (Chess game data format)
- JSON – JavaScript Object Notation (data format)
- FastAPI- python web farmwork
- React- JavaScript library used for building user interfaces
- JS – JavaScript

1.4 Assumptions

- Users know how to play chess and understand the basic rules of the game.
- No user accounts or authentication is needed to play.
- Users have a computer to run a desktop browser to play on.

1.5 References

- JS library for interactive web browser chess boards
chessboardjs.com
- React is a library used to build user interfaces
<https://react.dev>
- Chess.js is a JS library that will be used to handle chess logic and validation of moves on the frontend of our web application.
- JSON is a JS Object Notation. That will be used for data formatting to store the game state and help AI to evaluate moves from the data.
<https://www.json.org>
- Minimax Algorithm will be used for the AI decision making to help make the best optimal next move.
- Visual Studio Code will be our main source code editor used for both the frontend and backend development of our web application. It supports JS, Python, and React.
- FastAPI is a Python web framework to help build APIs
<https://fastapi.tiangolo.com/>
- PGN is the standard format to help record chess games.
- Python-Chess is a python chess library used to parse PGN formats and validate moves on the backend of the application. It will help manage the board state and validate legal moves being made by AI. It will be used with our custom minimax algorithm for our AI.

2.0 Constraints

These will be the limitations of our web application, the things that the application will not do. It will help narrow down our scope to help have a better understanding of how the application will work.

2.0.1 Guest Mode

The application will have Guest Mode only. We will not provide a signup or login for users. It will not keep track of stats or records of users in our database.

2.0.2 No Mobile

The web application will only be done in a web browser and not allow any mobile devices to play the game.

2.0.3 No Save or Restore Game Points

The user will not be allowed to exit the game and restore the previous saved game being played. The game will need to be finished in one play through.

2.0.4 Multiplayer Online

The game will only host gamers locally against other users based on time constraints of the project completion time.

2.0.5 AI Difficulty Scaling

The AI will make the best possible moves against a user. However, the AI will not improve and learn from past games with users. There will be no difficulty scaling for the AI.

2.1 Environment

The Web based application will be a local environment, where users can play locally against one another or AI engine to play against. Due to time constraints, Later improvements are games can be played over the internet and adding difficulty level of the ai engine.

2.2 System

Utilizing full stack development, there will be a user interface running on a web browser using react library from JavaScript, and back-end development will be built on python and open-source libraries for handling game state, game logic, player moves, and system processing.

3.0 Functional Requirements

3.1 Game Modes

3.1.1 User vs User Mode

- The system will allow two human players to play on the same machine.
- The system should switch between white and black chess pieces depending on whether it's their turn.

3.1.2 User vs AI Mode

- The system will allow human controlled players to participate against AI.
- The AI will generate movement based on the engine, using (Minimax algo, etc.)
- The system should allow the player to pick their colors(optional)

3.2 Game Rules

3.2.1 Standard Rules

- The system will enforce rules based on if the rules are legal in chess, this includes each individual piece movement and capture.

3.2.2 Special Rules

3.2.2.1 Castling

- The system will allow castling depending on if neither king nor rook have moved from their previous position and no check conditions prevent it. No pieces should be blocking king and rook.

3.2.2.2 Pawn Promotion

- The system should allow pawns to be promoted to a queen, rook, knight, or bishop, when they reach the end side of the board.

3.2.2.3 En Passant

- The system should implement En passant capture is when a pawn moves two squares, and an enemy pawn is in the position.

3.2.2.4 Double Step Pawn Move

- The system will allow pawns to move forward two squares from their initial rank

3.3 Database

3.3.1 PGN game state

- The system will store game history in Portable Game Notation format

3.3.2 Player's moves

- The system will log and track each player's moves.

3.3.3 AI engine data

- The system should store the evaluation AI data such as search depth and evaluated positions (optional)

3.3.4 AI generated moves

- The system should log the AI's selected movements for analysis

3.4 UI

3.4.1 Chessboard

3.4.1.1 Drag & drop pieces

- The system should support drag and drop pieces for movement

3.4.1.2 Checkmate, Stalemates Check for win condition

- The system will automatically detect all game ending conditions like checkmate, stalemate to progress games.

3.4.1.3 Player's turn (white or black)

- The system should designate a clear outline to indicate which player's turn it is.

3.4.1.4 Win/loss score counter

- The system should display some sort of scoreboard to show all outcomes and update some player statistics.

3.5 AI Engine

- Min-Max Algorithm
- Machine Learning Training
- Move generation

4.0 Non-Functional Requirements

4.1 Maintainability

The code will use Object-Oriented Programming, modular, and be commented appropriately to keep proper organization.

4.2 Reliability

The game does not make illegal moves and enforces the rules of the game of chess. The game should not encounter any fatal bugs that will crash or disconnect them from their matches.

4.3 Usability

Provide an easy to use and understandable interface for users to navigate. Users should be able to move pieces with ease if it's a legal move. Invalid moves should reset the piece back to its original position, and prompt or indicate to user that the move was invalid.

4.4 Scalability

We will design the web application and source code to be able to scale with any future addons. Including things like adding a signup and login feature for users that will keep track of their record, past games, and ELO ranking. As well as accommodating for a potential larger user base using the application. Also, potentially improving the AI to provide different difficulty settings that the user can play against.

5.0 External Interface Requirements

5.1 User Interface Requirements

- Menu to select between the different game modes
- GUI must have a chess board with drag and drop or click to move functionalities.

5.2 Hardware Interface Requirements

- Trackpad or mouse to move the chess pieces on the board.

5.3 Software Interface Requirements

- Database Storage- JSON file format to store PGN game data, user moves, and AI training data
- Frontend- Web browser for user interaction with applications.
- Backend – Python (Fast API) to communicate with AI engines

5.4 Communication Interface Requirements

The system communicates using PGN (Portable Game Notation) files combined with the JSON data format for storing and exchanging game data. User input (move selection) shall be converted into a PGN-compatible JSON file, which is then passed to the backend to update the game state. The backend processes the move, updates the game state, and returns the updated state in JSON format for the user interface. The system ensures synchronization between PGN storage and JSON-based communication so that saved and replayed games always reflect the same state as live gameplay.

6.0 Analysis

6.1 Use Cases

6.1.1 Player vs AI

- **Actors:** User, System (AI Engine)
- **Precondition:** Game initialized, board visible, user selects Player vs AI and chooses a side (white or black).
- **Flow:**
 - User makes a move on the board.
 - System converts input to PGN/JSON, validates if move is legal, and updates the game state.
 - AI Engine processes the new state and generates its move.
 - System updates the board with the AI's move.
 - Steps repeat until checkmate, stalemate, draw, or resignation.
- **Postcondition:** Game concludes with a recorded result and final state.

6.1.2 Use Case: Player vs Player (local)

- **Actors:** Two Users, System
- **Precondition:** Game initialized, board visible, both players present.
- **Main Flow:**
 - Players alternate moves,
 - system validates
 - updates the game state after each move.
- **Postcondition:** Game ends with checkmate, stalemate, or draw, and final state

6.2 Data Flow

6.2.1 Game Flow

- Input: User moves a chess piece on the UI chessboard.
- Process:
 - System converts the player's move into PGN/JSON format.
 - Backend validates the move legality.
 - System updates and stores the game state.
- Output: Updated game state displayed on the UI chessboard.

6.2.2 AI Engine Flow

- Input: Updated game state (PGN/JSON).
- Process:
 - AI Engine evaluates the position using the Mini-Max algorithm and the ML model.
 - AI generates its chess move.
 - Output: The AI's move is applied to the game state and displayed on the UI chessboard.

6.2.3 Training Flow

- Input: PGN game records (e.g., from Chess.com database API).
- Process:
 - Data pipeline parses game records into tensor features.
 - Training data is fed into a PyTorch training script for model evaluation and optimization.
- Output: A trained model with weights saved into the Chess AI program.